

FACULDADE DE TECNOLOGIA DE FRANCA – FATEC

CURSO: Desenvolvimento de Software Multiplataforma

Projeto Interdisciplinar (PI)

Gestão Ágil de Projetos de Software – 2026/1

Prof. Luiz Pedro Borges Júnior

Aluno: Jose Paulo Archetti Conrado

RA: 1091392523042

Cidade: Franca – Estado: São Paulo

Ano: 2026

Documentação: Desenvolvimento Software – AcademyFlow

Sumario

PARTE 1: Termo de abertura de Projeto (TAP) - Página 2

PARTE 2: Kanban - Pagina 11

PARTE 3: Scrum - Página 27

PARTE 1: Termo de abertura de Projeto (TAP)

Nome do Projeto: AcademyFlow – Gerenciamento de Eventos Acadêmicos.

Gestor do Projeto: Jose Paulo Archetti Conrado

Orçamento: R\$ 50.000,00

Prazo do projeto: 07/06/2026

Última revisão da TAP: 07/06/2026

JUSTIFICATIVA DO PROJETO

O gerenciamento de eventos acadêmicos e emissão de certificados ainda é feito de forma manual em muitas instituições, o que gera retrabalho e inconsistências. O AcademyFlow busca centralizar inscrições, presenças e certificados em uma plataforma moderna, reduzindo custos administrativos e aumentando a confiabilidade dos dados.

OBJETIVOS

- Desenvolver uma aplicação web responsiva para gestão de eventos acadêmicos.
 - Automatizar o registro de presença com QR Code.
 - Emitir certificados digitais validados.
 - Integrar com bancos de dados não relacionais (MongoDB).
- Reduzir em 40 % o tempo de gestão de eventos e aumentar a confiabilidade dos registros em 30 %.

PRODUTO E REQUISITOS

Produto: Plataforma web AcademyFlow

front-end em Next.js, back-end com Express/Prisma/MongoDB.

Requisitos principais:

- Cadastro de eventos, palestrantes e participantes.
- Registro de presença via QR Code (futura implementação).
 - Emissão automática de certificados digitais.
- Painel administrativo com relatórios e estatísticas.
- Integração com sistemas acadêmicos existentes (futura implementação).

MARCOS	PREMISSAS
• Início do desenvolvimento: 01/05/2026 • Entrega do protótipo funcional: 07/06/2026 • Testes de integração e validação: 07/06/2026 • Implantação piloto em instituição parceira: 15/01/2027 • Lançamento oficial da plataforma: 31/05/2027	• A equipe de desenvolvimento estará disponível em tempo integral durante o ciclo principal. • O ambiente de nuvem (Vercell) será contratado previamente. • As instituições parceiras fornecerão acesso aos dados acadêmicos necessários. • O projeto seguirá metodologias ágeis (Scrum) com sprints quinzenais.

EQUIPE		
Membro	Cargo	Responsabilidade
Jose Paulo Archetti Conrado	Product Owner	Gerenciar o desenvolvimento
Jose Paulo Archetti Conrado	Scrum Master	Implementar o desenvolvimento
Jose Paulo Archetti Conrado	Desenvolvedor	Desenvolver o produto

RESTRIÇÕES	RISCOS
• O orçamento total não poderá ultrapassar R\$ 50.000,00. • O prazo máximo de entrega é maio/2027. • O sistema deverá operar exclusivamente em ambiente web (sem versão mobile nativa). • A integração com bancos de dados externos dependerá de autorização institucional. • O design seguirá o padrão de identidade visual do Manual de Identidade Visual.	• • Técnico: Falhas na integração entre MongoDB. → Mitigação: realizar testes de carga e redundância antes da implantação. • • Operacional: Atrasos na entrega de dados pelas instituições parceiras. → Mitigação: estabelecer cronograma de compartilhamento e reuniões semanais. • • Financeiro: Custos adicionais com infraestrutura cloud. → Mitigação: monitorar consumo e ajustar escalabilidade conforme demanda. • • Humano: Rotatividade de membros da equipe. → Mitigação: documentar processos e manter repositório atualizado no GitHub.

- • Legal: Questões de LGPD na manipulação de dados de alunos. → Mitigação: implementar anonimização e consentimento explícito.

1.1 Perguntas e Respostas

☒ Por que o projeto é necessário?

O projeto é necessário porque o gerenciamento de eventos acadêmicos ainda é feito de forma manual em muitas instituições, gerando retrabalho, inconsistências e baixa confiabilidade dos dados

☒ Como o projeto ajudará a resolver os problemas identificados pela organização?

O projeto resolve esses problemas ao centralizar todo o fluxo de gestão (inscrições, presença e certificados) em uma única plataforma digital, automatizando processos e reduzindo erros humanos, além de facilitar a organização e consulta das informações.

☒ Qual a solução recomendada?

A solução recomendada é o desenvolvimento de uma **plataforma web (AcademyFlow)** com:

- Gestão completa de eventos, usuários e inscrições
- Registro de presença (com QR Code futuramente)
- Emissão automática de certificados digitais
- Painel administrativo com relatórios
- Integração com banco de dados (MongoDB)

☒ Quais os benefícios esperados?

Os principais benefícios são:

- Redução de aproximadamente **40% no tempo de gestão de eventos**
- Aumento de **30% na confiabilidade dos dados**

- Diminuição de retrabalho administrativo
- Organização centralizada das informações
- Maior eficiência operacional

☒ **O que vai ou pode ocorrer ao negócio, se a solução não for implantada?**

Caso não seja implantada:

- Continuidade de processos manuais e ineficientes
- Aumento de erros e inconsistências nos dados
- Maior custo operacional
- Dificuldade de controle e consulta de informações
- Perda de produtividade na gestão de eventos

☒ **Quando a solução será implantada? E os recursos necessários?**

Implantação:

- Protótipo funcional: **07/06/2026**
- Implantação piloto: **15/01/2027**
- Lançamento oficial: **31/05/2027**

Recursos necessários:

- Orçamento: **R\$ 50.000,00**
- Equipe (Product Owner, Scrum Master e Desenvolvedor)
- Tecnologias: Next.js, Node.js, Express, Prisma, MongoDB
- Infraestrutura em nuvem (Vercel)
- Metodologia ágil (Scrum)

1.2 TAP – AcademyFlow

1. Identificação do projeto

- Nome: AcademyFlow
- Tipo: sistema web para gerenciamento de eventos acadêmicos
- Estrutura do repositório: back-end, front-end e docs
- Tecnologias principais: Node.js, Express, Prisma, MongoDB, Next.js, TypeScript e Tailwind CSS
- Período de execução observada no repositório: 26/05/2026 a 28/05/2026
- Ordem real de desenvolvimento: back-end primeiro, front-end depois

2. Contexto e problema

O projeto nasceu para centralizar o fluxo de gestão de eventos acadêmicos, incluindo autenticação, cadastro de usuários, eventos, palestrantes, sessões, inscrições, presença e certificados. Antes da solução, essas atividades tendiam a ficar espalhadas ou dependentes de controles manuais.

3. Justificativa

O sistema melhora a organização do processo de inscrição e acompanhamento de participação em eventos acadêmicos, reduz retrabalho e facilita a consulta de dados por perfis distintos de usuário.

4. Objetivo geral

Entregar uma plataforma web completa para planejamento, operação e consulta de eventos acadêmicos, com back-end REST, front-end responsivo e suporte aos perfis administrador e participante.

5. Objetivos específicos

- Implementar autenticação com controle de sessão.
- Disponibilizar CRUDs para usuários, eventos, palestrantes, sessões, inscrições, presença e certificados.
- Permitir upload e consulta de certificados em PDF.
- Criar interface web para administração e área do participante.
- Validar os principais fluxos com testes automatizados.

6. Escopo

Em escopo

- Estrutura inicial do back-end com Express, Prisma e MongoDB.
- Camada de autenticação e autorização.
- Módulos de domínio do sistema: usuários, eventos, palestrantes, sessões, inscrições, presença e certificados.
- Estrutura inicial do front-end em Next.js.
- Área administrativa e área do participante.
- Testes unitários, de integração e E2E básicos.
- Documentação técnica de apoio.

Fora de escopo

- Aplicativo mobile nativo.
- Pagamentos online.
- Notificações por push ou SMS.
- Analytics avançado.
- Multi-tenant com diferentes instituições.

7. Entregáveis

- API REST funcional.
- Interface web funcional.
- Base de dados modelada em Prisma.
- Suíte mínima de testes.
- Documentação de planejamento e apoio.

8. Stakeholders

- Cliente/Professor orientador: valida escopo, cronograma e documentação.
- Equipe de desenvolvimento: implementa e testa as funcionalidades.
- Administrador do sistema: gerencia eventos e usuários.
- Participante: consulta eventos, se inscreve, acompanha presença e certificação.

9. Premissas

- O back-end serve como base para o front-end.
- O projeto deve refletir a sequência real dos commits do repositório.
- O front-end consome a API via credenciais em cookie.
- Os dados principais estão modelados em MongoDB com Prisma.

10. Restrições

- Prazo curto de desenvolvimento.
- Necessidade de manter consistência entre commits, código e documentação.
- Dependência de uma API pronta para o front-end.
- Validação acadêmica baseada em material do curso e em demonstração funcional.

11. Riscos

- Desalinhamento entre a documentação e a ordem real de entrega.
- Mudanças de escopo no meio do desenvolvimento.
- Falhas de integração entre front-end e back-end.
- Inconsistências de dados ou permissão por perfil.

12. Macro cronograma

Macrocronograma

Fase	Conteúdo	Dependência
Fase 1	Inicialização do back-end	Início do projeto
Fase 2	Prisma, MongoDB e schema de dados	Fase 1
Fase 3	Autenticação e CRUDs principais do back-end	Fase 2
Fase 4	Testes, ajustes e consolidação do back-end	Fase 3
Fase 5	Inicialização do front-end	Fase 4
Fase 6	Layout, autenticação e navegação	Fase 5
Fase 7	Área administrativa e área do participante	Fase 6
Fase 8	Testes E2E, refinamentos e documentação final	Fase 7

13. Critérios de aceite

- Login e logout funcionais.
- CRUDs principais operando com dados persistentes.
- Rotas protegidas por perfil.
- Area administrativa e área do participante acessíveis.
- Testes executáveis e documentação organizada.

PARTE 2: Kanban

2.1 Cards do Trello/Jira com checklists detalhados

2.1.1 - Back-end

BE-01 - Inicializar API Express

- [] Criar repositório e estrutura de pastas (src, routes, controllers)
- [] Configurar `package.json` com scripts (`dev`, `start`)
- [] Adicionar middlewares: CORS, cookie-parser, morgan, express.json
- [] Criar rota index e rota de health-check
- [] Testar servidor localmente em `PORT` configurada
- [] Documentar pontos de entrada no README

BE-02 - Integrar Prisma com MongoDB

- [] Instalar Prisma e dependências
- [] Adicionar `prisma/schema.prisma` com provider mongodb
- [] Configurar `DATABASE\_URL` no `.env.example`
- [] Implementar `src/database/client.js` (Prisma client)
- [] Rodar `prisma generate` e validar conexão
- [] Documentar comandos de migração/seed

BE-03 - Definir schema de domínio

- [] Modelar entidades: User, Speaker, Event, EventSession, Registration, Attendance, Certificate
- [] Definir relacionamentos e constraints no Prisma
- [] Validar modelos com `prisma validate`
- [] Gerar scripts de seed com dados basicos
- [] Documentar modelo no README ou arquivo de modelagem

BE-04 - Implementar autenticação

- [] Definir endpoints: `POST /auth/login`, `POST /auth/logout`, `POST /auth/refresh`, `GET /auth/me`
- [] Implementar JWT + cookie secure, ou strategy escolhida
- [] Hash de senhas com `bcryptjs` no registro/seed
- [] Testes unitários para fluxo de login/logout

- [] Documentar contrato de autenticação (ex.: cookies, cabeçalhos)

BE-05 - Criar gestão de usuários

- [] Endpoints CRUD: `GET /users`, `POST /users`, `GET /users/:id`, `PUT /users/:id`, `DELETE /users/:id`
- [] Validações de campos e regras de negócio
- [] Permissões: apenas admin pode criar ou deletar usuários (quando aplicável)
- [] Testes de integração cobrindo principais cenários
- [] Documentar exemplos de request/response

BE-06 - Criar gestão de eventos

- [] Endpoints CRUD para eventos
- [] Validar datas, limites e integridade referencial
- [] Associar palestrantes e sessões quando necessário
- [] Testes para criação, atualização e exclusão
- [] Documentar payloads e exemplos

BE-07 - Criar gestão de palestrantes

- [] Endpoints CRUD para palestrantes
- [] Validações de dados (nome, bio, contatos)
- [] Testes de ciclo de vida (create->update->delete)
- [] Documentar uso nos endpoints de eventos

BE-08 - Criar gestão de sessões

- [] Endpoints para sessões vinculadas a eventos (`/events/:eventId/sessions`)
- [] Validações de horário, overlap e duração
- [] Testes de integração entre evento e sessoes
- [] Documentar como relacionar sessões aos eventos

BE-09 - Recalcular progresso de presença

- [] Implementar serviço para recalculo de progresso por inscricao/evento
- [] Atualizar no momento de CRUD de presença/inscrição
- [] Adicionar testes que confirmem o recalculo
- [] Documentar formula/critério de progresso

BE-10 - Criar gestão de inscrições

- [] Endpoints CRUD para inscrições
- [] Regras: única inscrição por usuário/sessão, limites de vagas
- [] Validações e mensagens de erro claras
- [] Testes transacionais para evitar duplicidade

BE-11 - Criar gestão de presença

- [] Endpoints para registrar e consultar presença por inscrição
- [] Validações de autenticidade e período
- [] Hooks para atualizar progresso e emitir eventos se necessário
- [] Testes cobrindo casos de borda

BE-12 - Criar gestão de certificados

- [] Endpoint para upload de PDF (`POST /certificates/upload`)
- [] Salvar metadados e URL publica/privada conforme requisitado
- [] Endpoints CRUD para certificados
- [] Testes para upload e download (mock de storage se necessário)

BE-13 - Criar testes de usuários e eventos

- [] Escrever testes unitários e de integração para fluxos de usuários
- [] Cobrir criação, autenticação e autorização
- [] Testes para criação e consulta de eventos

BE-14 - Criar testes de palestrantes e sessões

- [] Cobrir lifecycle de palestrantes e sessões
- [] Validar rejeições e IDs inválidos

BE-15 - Criar testes de inscrição e presença

- [] Testar regras de negocio: duplicidade, limites e recalcule de progresso
- [] Simular fluxos completos (inscrição -> presença -> certificado)

BE-16 - Ajustar script de seed e smoke tests

- [] Preparar massa mínima para testes E2E
- [] Implementar smoke test que valida endpoints principais
- [] Automatizar seed no workflow de teste se necessário

2.1.2 - Front-end

FE-01 - Inicializar Next.js

- [] Criar app Next.js com TypeScript
- [] Configurar `NEXT\_PUBLIC\_API\_BASE\_URL` e `.env.example`
- [] Rodar build inicial e validar server-side / client-side

FE-02 - Definir tema visual e componentes

- [] Criar biblioteca de componentes: Button, Input, Card, Badge, Toast, Skeleton
- [] Configurar Tailwind e tema customizado
- [] Garantir tokens de estilo e responsividade

FE-03 - Criar integração com API

- [] Criar cliente HTTP central (fetch/axios) com tratamento de erros
- [] Tipar respostas usando tipos compartilhados/`domain.ts`
- [] Tratar autenticação via cookie/credenciais automaticamente

FE-04 - Implementar login e registro

- [] Implementar telas de login e cadastro com validação de formulário
- [] Persistir sessão e redirecionar após login
- [] Testar fluxo de autenticação manualmente e com E2E

FE-05 - Proteger rotas com AuthGate

- [] Implementar componente `AuthGate` para proteger rotas
- [] Respeitar roles (admin vs participant)
- [] Adicionar fallback de loading e mensagem de acesso negado

FE-06 - Criar shell do dashboard

- [] Implementar header, menu lateral e area de conteúdo
- [] Integrar `LogoutButton` e perfil do usuário
- [] Garantir navegação entre áreas admin/participant

FE-07 - Criar dashboard do admin

- [] Montar cards de resumo (usuários, eventos, presenças, certificados)
- [] Links rápidos para CRUDs principais
- [] Testar responsividade e acessibilidade mínima

FE-08 - Criar CRUD de usuários no admin

- [] Listagem com paginação e filtros
- [] Formulário de criação/edição com validação
- [] Actions para bloquear/desbloquear e deletar

FE-09 - Criar CRUD de palestrantes no admin

- [] Listagem e formulário de palestrante
- [] Upload de foto/links opcionais

FE-10 - Criar CRUD de eventos no admin

- [] Listagem, criar/editar evento com definição de sessões
- [] Validação de datas e limites

FE-11 - Criar CRUD de inscrições no admin

- [] Filtrar por evento/usuário/sessão
- [] Editar status e gerenciar conflitos

FE-12 - Criar CRUD de sessões no admin

- [] Gerenciar sessões de cada evento, horários e salas

FE-13 - Criar CRUD de presença no admin

- [] Selecionar sessão e registrar presenças em lote
- [] Ferramenta para ajuste manual e correções

FE-14 - Criar CRUD de certificados no admin

- [] Emitir certificado, anexar PDF e gerenciar metadados
- [] Baixar e revogar certificados

FE-15 - Criar layout do participante

- [] Navegação simplificada para participante
- [] Acesso rápido a inscrições e certificados

FE-16 - Criar dashboard do participante

- [] Exibir progresso, próximos eventos e certificados

FE-17 - Criar tela de eventos do participante

- [] Busca, filtros, detalhes do evento e inscrição

FE-18 - Criar tela de inscrições do participante

- [] Exibir status, opções de cancelamento e acesso ao certificado

FE-19 - Criar tela de certificados do participante

- [] Listar certificados disponíveis e permitir download

FE-20 - Criar tela de perfil do participante

- [] Editar dados básicos, foto e preferencias

FE-21 - Criar testes E2E

- [] Escrever testes Playwright cobrindo fluxos críticos (admin e participant)
- [] Integrar testes no CI/localmente

FE-22 - Revisar UX e responsividade

- [] Ajustar estados de loading, empty states e mensagens de erro
- [] Testar em tamanhos de tela e corrigir regressões

Spaces

AcdmyFlow  ...

 Summary  List  **Board**  Development  Forms  Timeline  Docs  Reports +

 **JC**  Filter

IN PROGRESS

IN REVIEW

TO DO

+ Create

DONE 38 ✓

[BE] Inicializar API Express ...

back-end

16 May 2026

✓ AE-1 ✓ JC

[BE] Integrar Prisma com MongoDB

back-end,db

16 May 2026

✓ AE-2 ✓ JC

[BE] Definir schema de dominio

back-end,db

16 May 2026

✓ AE-3 ✓ JC

[BE] Implementar autenticao

back-end,auth

16 May 2026

✓ AE-4 ✓ JC

[BE] Criar gestao de usuarios

back-end,crud

16 May 2026

✓ AE-5 ✓ JC

[BE] Criar gestao de eventos

back-end,crud

16 May 2026

✓ AE-6 ✓ JC

[BE] Criar gestao de palestrantes

back-end,crud

16 May 2026

Spaces

AcademyFlow  ...

 Summary  List  **Board**  Development  Forms  Timeline  Docs  Reports +

Q Search board  **JC**  Filter

IN PROGRESS **IN REVIEW** **TO DO**

[BE] Criar gestao de eventos
back-end,crud
16 May 2026
☒ AE-6  **JC**

[BE] Criar gestao de palestrantes
back-end,crud
16 May 2026
☒ AE-7  **JC**

[BE] Criar gestao de sessoes ...
back-end,crud
16 May 2026
☒ AE-8  **JC**

[BE] Recalcular progresso de presenca
back-end
16 May 2026
☒ AE-9  **JC**

[BE] Criar gestao de inscricoes
back-end,crud
16 May 2026
☒ AE-10  **JC**

[BE] Criar gestao de presenca
back-end,crud
16 May 2026
☒ AE-11  **JC**

[BE] Criar gestao de certificados
back-end,crud
16 May 2026
☒ AE-12  **JC**

Spaces

AcdmyFlow  ...

Summary List **Board** Development Forms Timeline Docs Reports +

Q Search board  Filter

IN PROGRESS	IN REVIEW	TO DO
		[BE] Criar testes de usuarios e eventos back-end,testes 16 May 2026 ☒ AG-13  
		[BE] Criar testes de palestrantes e sessoes back-end,testes 16 May 2026 ☒ AG-14  
		[BE] Criar testes de inscricao e presenca back-end,testes 16 May 2026 ☒ AG-15  
		[BE] Ajustar script de seed e smoke tests back-end,testes 16 May 2026 ☒ AG-16  
		[FE] Inicializar Next.js ... front-end 31 May 2026 ☒ AG-17  
		[FE] Definir tema visual e componentes front-end,ui 31 May 2026 ☒ AG-18  
		[FE] Criar integracao com API front-end 31 May 2026 ☒ AG-19  

Spaces

AcdmyFlow  ...

 Summary  List  **Board**  Development  Forms  Timeline  Docs  Reports +

 **JC**  Filter

IN PROGRESS	IN REVIEW	TO DO
		☒ AG-18   [FE] Criar integracao com API front-end 31 May 2026
		☒ AG-19   [FE] Implementar login e registro front-end,auth 31 May 2026
		☒ AG-20   [FE] Proteger rotas com AuthGate front-end,auth 31 May 2026
		☒ AG-21   [FE] Criar shell do dashboard front-end,ui 31 May 2026
		☒ AG-22   [FE] Criar dashboard do admin front-end,ui 31 May 2026
		☒ AG-23   [FE] Criar CRUD de usuarios no admin front-end,crud 31 May 2026
		☒ AG-24   [FE] Criar CRUD de palestrantes no admin front-end,crud 31 May 2026
		☒ AG-25  

Spaces

AcdmyFlow  ...

 Summary  List  **Board**  Development  Forms  Timeline  Docs  Reports +

 **JC**  Filter

IN PROGRESS

IN REVIEW

TO DO

[FE] Criar CRUD de eventos no admin

front-end,crud

31 May 2026

☒ AG-26  

[FE] Criar CRUD de inscricoes no admin

front-end,crud

31 May 2026

☒ AG-27  

[FE] Criar CRUD de sessoes no admin

front-end,crud

31 May 2026

☒ AG-28  

[FE] Criar CRUD de presenca no admin

front-end,crud

31 May 2026

☒ AG-29  

[FE] Criar CRUD de certificados no admin

front-end,crud

31 May 2026

☒ AG-30  

[FE] Criar layout do participante

front-end,ui

31 May 2026

☒ AG-31  

[FE] Criar dashboard do participante

front-end,ui

31 May 2026

☒ AG-32  

Spaces

AcademyFlow  ...

 Summary  List  **Board**  Development  Forms  Timeline  Docs  Reports +

 Search board  **JC**  Filter

IN PROGRESS

IN REVIEW

TO DO

front-end,ui

 31 May 2026

☒ **AE-32**  **JC**

[FE] Criar tela de eventos do participante

front-end,ui

 31 May 2026

☒ **AE-33**  **JC**

[FE] Criar tela de inscrições do participante

front-end,ui

 31 May 2026

☒ **AE-34**  **JC**

[FE] Criar tela de certificados do participante

front-end,ui

 31 May 2026

☒ **AE-35**  **JC**

[FE] Criar tela de perfil do participante

front-end,ui

 31 May 2026

☒ **AE-36**  **JC**

[FE] Criar testes E2E ...

front-end,testes

 31 May 2026

☒ **AE-37**  **JC**

[FE] Revisar UX e responsividade

front-end,ui

 31 May 2026

☒ **AE-38**  **JC**

2.2 - PERT / CPM do projeto

Este documento usa uma estimativa didática baseada na ordem real dos commits do repositório: primeiro o back-end, depois o front-end.

2.2.1 - PERT / CPM do back-end

Atividades e dependências

1, PERT / CPM do back-end

ID	Atividade	Duração (dias)	Predecessora
A	Inicialização do servidor Express	1	–
B	Integração Prisma + MongoDB	1	A
C	Schema de dominio	1	B
D	Autenticação e usuários	1	C
E	Eventos	1	D
F	Palestrantes	1	E
G	Sessões de evento	1	F
H	Inscrições	1	G
I	Presença	1	H
J	Certificados	1	I
K	Testes e estabilização	1	J

Caminho crítico do back-end

A -> B -> C -> D -> E -> F -> G -> H -> I -> J -> K

Duração total estimada do back-end: 11 dias

Diagrama PERT / CPM do back-end (Ver página 26)

2.2.2 - PERT / CPM do front-end

Atividades e dependências

2. PERT / CPM do front-end

ID	Atividade	Duração (dias)	Predecessora
L	Inicialização do Next.js	1	K
M	Base visual e componentes	1	L
N	Integração com API e tipos	1	M
O	Login, registro e sessão	1	N
P	AuthGate e shell do dashboard	1	O
Q	Área administrativa	2	P
R	Área do participante	2	P
S	Testes E2E e refinamentos	1	Q, R

Caminho crítico do front-end

L -> M -> N -> O -> P -> Q -> S

Obs.: a trilha do participante (R) entra em paralelo com a área administrativa (Q), mas o fechamento depende das duas.

Duração total estimada do front-end: 8 dias

Diagrama do front-end (Ver página 26)

2.2.3 - Visão consolidada do projeto

3. Visão consolidada do projeto

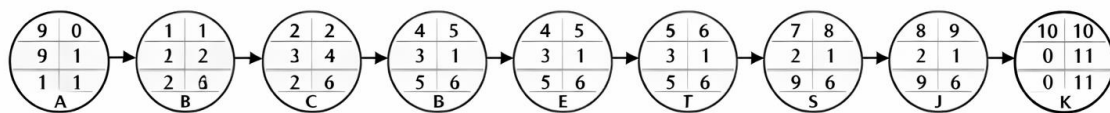
Etapa	Duração estimada	Observação
Back-end	11 dias	Base de dados, autenticação e módulos de domínio
Front-end	8 dias	Interface, dashboards e área do participante
Fechamento	1 dia	Testes finais e consolidação da documentação

Caminho crítico consolidado

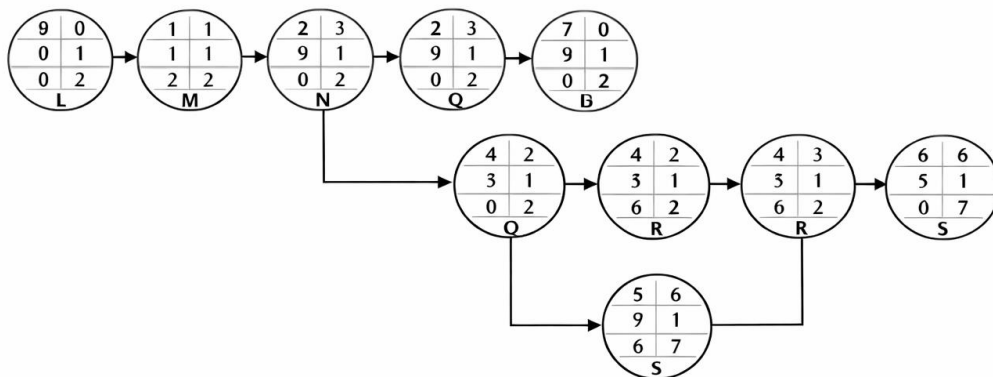
A -> B -> C -> D -> E -> F -> G -> H -> I -> J -> K -> L -> M -> N -> O -> P -> Q -> S

2.2.4 - Diagrama PERT / CPM

Back-end (11 dias)



Front-end (8 dias)



PARTE 3: Scrum

3.1 - Scrum no Projeto AcademyFlow

Com base na atividade de Scrum da disciplina e no backlog já estabelecido para o projeto AcademyFlow, este documento organiza a apresentação do projeto em formato de Scrum. A proposta simula como a equipe teria planejado e acompanhado o desenvolvimento em sprints, mesmo com o projeto já concluído.

3.1.1 Nome do Projeto

- Nome do sistema: AcademyFlow
- Integrantes da equipe: equipe do projeto vinculada ao repositório atual
- Objetivo principal: disponibilizar uma plataforma web para gerenciamento de eventos acadêmicos, com área administrativa e área do participante

3.1.2. Declaração de Visão do Projeto

a) Objetivo

Centralizar o gerenciamento de eventos acadêmicos, cobrindo autenticação, cadastro, inscrições, presença, certificados e acompanhamento dos usuários.

b) Descrição

O sistema funciona como uma plataforma web completa, com back-end em API REST e front-end em Next.js, permitindo que administradores e participantes executem seus fluxos em uma interface responsiva e integrada.

c) Público-alvo

- Administradores do sistema
- Organizadores de eventos acadêmicos
- Participantes e inscritos nos eventos

d) Problema resolvido

O projeto reduz o uso de controles manuais e dispersos para cadastro, inscrição e acompanhamento de eventos acadêmicos, facilitando a organização e a consulta de informações.

e) Diferencial

O diferencial do AcademyFlow é a integração entre uma API estruturada, uma interface responsiva e o fluxo de acompanhamento de eventos com autenticação, permissão por perfil e suporte a certificados em PDF.

3.1.3. Mapeamento das Partes Interessadas

- Cliente / Professor orientador
- Equipe de desenvolvimento
- Administrador do sistema
- Participante dos eventos
- Usuário autenticado da plataforma

3.1.4. Product Backlog

O backlog abaixo reutiliza os cards já definidos no projeto e serviu de base para a criação das sprints.

Back-end

- Inicializar API Express
- Integrar Prisma com MongoDB
- Definir schema de domínio
- Implementar autenticação
- Criar gestão de usuários
- Criar gestão de eventos
- Criar gestão de palestrantes
- Criar gestão de sessões
- Recalcular progresso de presença
- Criar gestão de inscrições
- Criar gestão de presença
- Criar gestão de certificados
- Criar testes de usuários e eventos
- Criar testes de palestrantes e sessões
- Criar testes de inscrição e presença
- Ajustar script de seed e smoke tests

Front-end

- Inicializar Next.js
- Definir tema visual e componentes
- Criar integração com API
- Implementar login e registro
- Proteger rotas com AuthGate
- Criar shell do dashboard
- Criar dashboard do admin
- Criar CRUD de usuários no admin
- Criar CRUD de palestrantes no admin
- Criar CRUD de eventos no admin
- Criar CRUD de inscrições no admin
- Criar CRUD de sessões no admin
- Criar CRUD de presença no admin
- Criar CRUD de certificados no admin
- Criar layout do participante
- Criar dashboard do participante
- Criar tela de eventos do participante
- Criar tela de inscrições do participante
- Criar tela de certificados do participante
- Criar tela de perfil do participante
- Criar testes E2E
- Revisar UX e responsividade

3.1.5. Histórias de Usuário

a. Login com segurança

Como usuário, quero fazer login na plataforma para acessar as funcionalidades do meu perfil.

b. Gestão administrativa

Como administrador, quero cadastrar e editar usuários, eventos e palestrantes para manter o sistema atualizado.

c. Inscrição em eventos

Como participante, quero visualizar eventos disponíveis e me inscrever para garantir minha participação.

d. Acompanhamento de presença

Como administrador, quero registrar presença nas sessões para controlar a participação dos inscritos.

e. Certificados

Como participante, quero consultar e baixar meus certificados para comprovar minha presença nos eventos.

f. Proteção de acesso

Como usuário autenticado, quero acessar apenas as páginas permitidas ao meu perfil para manter a segurança do sistema.

3.1.6. Planejamento das Sprints

O planejamento abaixo define as sprints praticadas pelo desenvolvedor de back-end e front-end.

Sprint 1 - Estrutura e regras do back-end

- Objetivo da Sprint: estruturar a API, modelar os dados e entregar os CRUDs principais do domínio.
- Tempo estimado: 11 dias
- Funcionalidades priorizadas: Express, Prisma, MongoDB, schema de domínio, autenticação, usuários, eventos, palestrantes, sessões, inscrições, presença, certificados
- Responsável por tarefa: desenvolvedor de back-end

Sprint 2 - Interface e fluxos do front-end

- Objetivo da Sprint: entregar a experiencia visual, a navegação e os fluxos do administrador e do participante.
- Tempo estimado: 8 dias
- Funcionalidades priorizadas: Next.js, tema visual, integração com API, login, proteção de rotas, dashboard, CRUDs administrativos, area do participante, testes E2E e ajustes de UX
- Responsável por tarefa: desenvolvedor de front-end

3.1.7. Relatório Daily Scrums

Daily Scrums agendadas durante a execução das sprints.

Back-end (11 dailys – 16 cards)

Daily	O que foi feito	O que será feito	Impedimentos
1	Estrutura inicial da API Express	Integração com Prisma e MongoDB	Validação da conexão com banco
2	Definição do schema de domínio	Autenticação inicial	Dependência da modelagem para testes
3	Autenticação e gestão de usuários	CRUD de eventos	Ajustes de permissões administrativas
4	CRUD de palestrantes	CRUD de sessões	Validação de horários e sobreposição
5	CRUD de inscrições	Gestão de presença	Regras de negócio para evitar duplicidade
6	Gestão de presença	Gestão de certificados	Definição de metadados e armazenamento
7	Gestão de certificados	Testes de usuários e eventos	Massa de dados para validação
8	Testes de palestrantes e sessões	Testes de inscrição e presença	Ajustes em regras de negócio
9	Testes de inscrição e presença	Ajustes de script de seed	Dados consistentes para smoke tests
10	Ajustes de seed e smoke tests	Estabilização da API	Refinamentos finais
11	Estabilização da API e documentação	Preparação para front-end	Alinhamento de endpoints

- **Daily 1 – Início da Sprint Back-end**

O que foi feito: inicialização da API Express e configuração básica do servidor.

O que será feito: integração com Prisma e MongoDB.

Impedimentos: validação da conexão com banco antes de avançar.

- **Daily 2 – Continuidade da Sprint Back-end**

O que foi feito: definição do schema de domínio.

O que será feito: autenticação inicial e fluxo de login/logout.

Impedimentos: dependência da modelagem para testes de autenticação.

- **Daily 3 – Avanço da Sprint Back-end**

O que foi feito: autenticação e gestão de usuários.

O que será feito: CRUD de eventos.

Impedimentos: ajustes de permissões administrativas.

- **Daily 4 – Meio da Sprint Back-end**

O que foi feito: CRUD de palestrantes.

O que será feito: CRUD de sessões vinculadas a eventos.

Impedimentos: validação de horários e sobreposição de sessões.

- **Daily 5 – Consolidação da Sprint Back-end**

O que foi feito: CRUD de inscrições.

O que será feito: gestão de presença.

Impedimentos: dependência de regras de negócio para evitar duplicidade.

- **Daily 6 – Continuidade da Sprint Back-end**

O que foi feito: gestão de presença.

O que será feito: gestão de certificados.

Impedimentos: definição de metadados e armazenamento seguro de PDFs.

- **Daily 7 – Avanço da Sprint Back-end**

O que foi feito: gestão de certificados.

O que será feito: testes de usuários e eventos.

Impedimentos: necessidade de massa de dados para validação.

- **Daily 8 – Meio da Sprint Back-end**

O que foi feito: testes de palestrantes e sessões.

O que será feito: testes de inscrição e presença.

Impedimentos: ajustes em regras de negócio e duplicidade.

- **Daily 9 – Consolidação da Sprint Back-end**

O que foi feito: testes de inscrição e presença.

O que será feito: ajustes de script de seed.

Impedimentos: dependência de dados consistentes para smoke tests.

- **Daily 10 – Finalização da Sprint Back-end**

O que foi feito: ajustes de seed e smoke tests.

O que será feito: estabilização da API.

Impedimentos: nenhum crítico, apenas refinamentos.

- **Daily 11 – Encerramento da Sprint Back-end**

O que foi feito: estabilização da API e documentação técnica.

O que será feito: preparação para início do front-end.

Impedimentos: alinhamento de endpoints para consumo pelo front-end.

Front-end (8 dailys – 22 cards)

Daily	O que foi feito	O que será feito	Impedimentos
1	Inicialização do Next.js e tema visual	Integração com API e autenticação	Dependência da API estabilizada
2	Login, registro e AuthGate	Shell do dashboard	Ajustes de sessão e cookies
3	Dashboard do admin	CRUD de usuários e palestrantes	Validação de formulários e permissões
4	CRUD de eventos e inscrições	CRUD de sessões e presença	Dependência de dados do back-end
5	CRUD de certificados	Layout do participante	Navegação e responsividade
6	Dashboard do participante	Tela de eventos e inscrições	Refinamento de filtros e busca
7	Telas de certificados e perfil	Testes E2E	Massa de dados para simulação
8	Testes E2E, revisão de UX	Apresentação final e documentação	Nenhum crítico

- **Daily 1 – Início da Sprint Front-end**

O que foi feito: inicialização do Next.js e definição do tema visual.

O que será feito: integração com API e autenticação.

Impedimentos: dependência da API estabilizada.

- **Daily 2 – Continuidade da Sprint Front-end**

O que foi feito: login, registro e AuthGate.

O que será feito: shell do dashboard.

Impedimentos: ajustes de sessão e cookies.

- **Daily 3 – Avanço da Sprint Front-end**

O que foi feito: dashboard do admin.

O que será feito: CRUD de usuários e palestrantes.

Impedimentos: validação de formulários e permissões.

- **Daily 4 – Meio da Sprint Front-end**

O que foi feito: CRUD de eventos e inscrições.

O que será feito: CRUD de sessões e presença.

Impedimentos: dependência de dados do back-end para testes.

- **Daily 5 – Consolidação da Sprint Front-end**

O que foi feito: CRUD de certificados.

O que será feito: layout do participante.

Impedimentos: ajustes de navegação e responsividade.

- **Daily 6 – Continuidade da Sprint Front-end**

O que foi feito: dashboard do participante.

O que será feito: tela de eventos e inscrições.

Impedimentos: refinamento de filtros e busca.

- **Daily 7 – Avanço da Sprint Front-end**

O que foi feito: telas de certificados e perfil do participante.

O que será feito: testes E2E.

Impedimentos: necessidade de massa de dados para simulação.

- **Daily 8 – Encerramento da Sprint Front-end**

O que foi feito: testes E2E, revisão de UX e responsividade.

O que será feito: apresentação final e consolidação da documentação.

Impedimentos: nenhum crítico no encerramento.

3.1.8. Valores do Scrum no Projeto

Coragem

A equipe assumiu a implementação de uma stack completa, do back-end ao front-end, enfrentando integração, autenticação e regras de negócio.

Foco

Cada sprint concentrou o trabalho em um conjunto claro de entregas: primeiro o back-end, depois o front-end.

Comprometimento

O backlog foi executado de forma sequencial e os testes foram incluídos para consolidar as entregas.

Respeito

As funcionalidades foram organizadas para atender administradores e participantes, mantendo perfis e permissões distintas.

Abertura

A documentação foi mantida alinhada com a ordem real do desenvolvimento, permitindo revisão e apresentação transparente ao professor.